ai_client.py - Talking to the AI Provider

What this file does (big picture)

ai_client.py owns the technical connection to an AI provider.

Business jobs:

1) Read which provider/model to use from .env
2) Build a client that can complete a chat
3) Hide OpenAI SDK details from the rest of the app
4) Make it easy to add Anthropic (or others) later behind the same shape

This file does NOT decide what to say to customers yet.
responses.py will call it later when we turn templates into AI wording.

New words you will see in this file

adapter / wrapper	A thin layer that translates our app's needs into a vendor's SDK.
Protocol	A Python interface-by-shape: any class with complete() can be an AiClient.
ChatMessage	One turn: system, user, or assistant text.
factory	get_ai_client() builds the right client from configuration.
temperature	How creative the model is; lower = more steady/factual.

Flow while the program runs

.env (AI_PROVIDER, OPENAI_API_KEY, OPENAI_MODEL) -> get_ai_client() ->
OpenAiClient.complete(messages) -> text string for responses.py later.

ai_client.py - Talking to the AI Provider

1-16

```
"""AI provider client..."""
```

Plain English:
Docstring: one place for connection details; secrets stay in config/.env.

18-24

```
imports + env constant names
```

Plain English:
os for settings; Protocol for the interface; config for the API key.

27-30

```
DEFAULT_PROVIDER / DEFAULT_OPENAI_MODEL
```

Plain English:
Sensible defaults so .env only needs the key to start.

33-45

```
@dataclass ChatMessage
```

Plain English:
role + content. system = standing rules; user = question; assistant = prior reply.

48-62

```
class AiClient(Protocol)
```

Plain English:
The promise every provider must keep: provider_name, model_name, complete().

65-66

```
class AiClientError
```

Plain English:
Clear error type when the key is missing or the API call fails.

69-97

```
class OpenAiClient
```

Plain English:
Wraps the official OpenAI SDK. Import is inside __init__ so other providers are optional.

79-97

```
def complete(...)
```

Plain English:
Converts ChatMessage list to the OpenAI payload, calls chat.completions.create, returns text.

100-107

```
_selected_provider / _selected_openai_model
```

Plain English:
Read AI_PROVIDER and OPENAI_MODEL from the environment after load_env().

110-132

```
def get_ai_client()
```

Plain English:
Factory: today builds OpenAiClient; unknown providers raise a helpful AiClientError.

135-141

```
def describe_ai_client()
```

Plain English:
Safe startup line: provider + model, never the API key.

ai_client.py - Talking to the AI Provider

144-157

```
def smoke_complete(...)
```

Plain English:

Optional live ping to prove the connection. Costs a tiny amount - use intentionally.